

Playlist Generator

Temas Avanzados de Deep Learning

Grupo 3

Nicolás Rossi

Axel Castro Benza

Lucas David Perri

2026



00

Introducción

Por qué el problema es relevante

Introducción

Las plataformas actuales priorizan historial de escucha y popularidad, no la intención expresada en lenguaje natural

- Recomiendan sobre todo por lo que ya escuchaste y por tendencias globales.
- No interpretan pedidos con contexto (“para estudiar”, “sin reggaetón”, “de noche”).



01

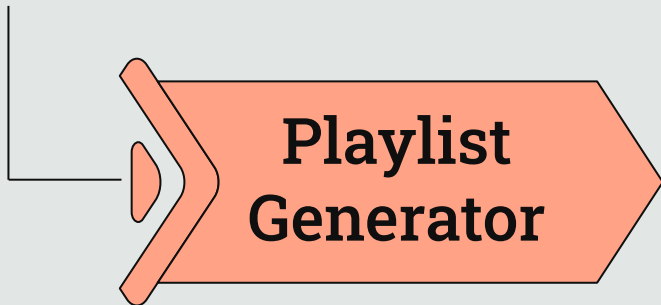
El Problema

Generar playlists a partir de una consulta de NL

El Problema

Natural Language Query:

"Rock and country for a long road trip"



Generated Playlist:

King of the Road - George Claymon
Another Brick in the Wall - Pink Floyd
Ring of Fire - Johnny Cash
I Got Your Country Right Here -
Gretchen Wilson
Sweet Home Alabama - Lynyrd
Skynyrd
Forever and Ever, Amen - Randy
Travis
I Love Rock 'N Roll - Joan Jett & the
Blackhearts

Description:

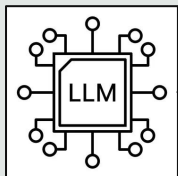
"This playlist perfectly blends rock and country, creating an upbeat and adventurous vibe ideal for a long road trip..."

02

¿Por qué RAG?

Problemas que el modelo sin retrieval no resuelve

¿Por qué un LLM Solo No Basta?



1.

No conoce todas las canciones (no tiene indexadas ~4.500 canciones con metadatos, letras y features) por lo que puede inventar títulos o artistas.

2.

No hace retrieval eficiente (vectores + filtros) sobre miles de tracks.

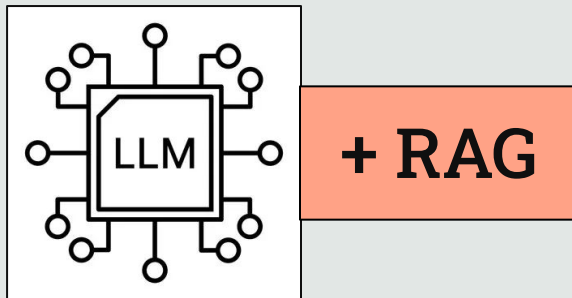
3.

No aplica reglas duras (duración, “sin reggaetón”, artistas excluidos, etc.).

4.

No garantiza trazabilidad, difícil auditar de dónde salió cada canción ni medir calidad (faithfulness, context recall).

¿Por qué RAG?



Con RAG se recuperan candidatos reales del índice, se rankean, se secuencian y el LLM termina explicando con contexto verificable.

03

Desafíos Encontrados

Problemas hallados a la hora de construir el dataset

Spotify Audio Features API - Deprecated

Web API endpoint integration

Effective today, **new** Web API use cases will no longer be able to access or use the following endpoints and functionality in their third-party applications. Applications with existing extended mode Web API access that were relying on these endpoints remain unaffected by this change.

1. [Related Artists](#)
2. [Recommendations](#)
3. [Audio Features](#)
4. [Audio Analysis](#)
5. [Get Featured Playlists](#)
6. [Get Category's Playlists](#)
7. [30-second preview URLs](#), in multi-get responses (SimpleTrack object)
8. Algorithmic and Spotify-owned editorial playlists

Campos utilizados a la hora de describir la canción (junto a letras + metadata)



• RESPONSE SAMPLE

```
1  {
2    "acousticness": 0.00242,
3    "analysis_url":
4      "https://api.spotify.com/v1/audio-
5      analysis/2takcw0aAZWiXQijPHIx7B",
6    "danceability": 0.585,
7    "duration_ms": 237040,
8    "energy": 0.842,
9    "id": "2takcw0aAZWiXQijPHIx7B",
10   "instrumentalness": 0.00686,
11   "key": 9,
12   "liveness": 0.0866,
13   "loudness": -5.883,
14   "mode": 0,
15   "speechiness": 0.0556,
16   "tempo": 118.211,
17   "time_signature": 4,
18   "track_href":
19     "https://api.spotify.com/v1/tracks/2takcw0aAZWiXQijPHIx7B",
20   "type": "audio_features",
21   "uri":
22     "spotify:track:2takcw0aAZWiXQijPHIx7B",
23   "valence": 0.428
24 }
```

Solución Implementada

Recreamos el esquema de Spotify. Con features extraídas **localmente** utilizando Essentia + MusiCNN sobre archivos WAV.

Compromisos Asumidos:

Liveness defaultea a **0.0** y **Time_Signature** a **4** porque Essentia no proveía una manera directa de cálculo o inferencia.

Modelos Pre-Entrenados utilizados:

- emomusic → **valence + arousal** (mood)
- voice_instrumental → **instrumentalness**
- mood_acoustic → **acousticness**

Essentia

Open-source library and tools for audio and music analysis, description and synthesis

Costo y Tiempo de Indexado

```
(venv) axelcastro@MacBook-de-Axel playlist-generator % python -m playlist_rag.cli.index \
    --parquet data/unified_tracks.parquet
22:47:16 INFO playlist_rag.indexing.pipeline: Skip set built: 20 tracks already complete
Indexing: 4495track [4:45:30, 3.81s/track]
03:32:47 INFO playlist_rag.cli.index: Done. RunStats(total=4495, skipped=20, succeeded=4475, failed=0, elapsed=17131.4s, rate=0.26 tracks/s)
RunStats(total=4495, skipped=20, succeeded=4475, failed=0, elapsed=17131.4s, rate=0.26 tracks/s)
```

El proceso de indexación implica procesar 4495 canciones, realizando para cada una descripción mediante un LLM y el cálculo de embeddings. Resultando en una ejecución de varias horas.

Unificación Datasets y Búsqueda de Lyrics

1243

De las 4495 canciones presentes en el dataset de Spotify, solo 1243 pudieron ser vinculadas exitosamente con el dataset de Genius (mediante Fuzzy Join).

828

De las canciones restantes sin letras se recuperaron 828 letras gracias a lyrics.ovh.

**2071 canciones con
letras (~45%)**

Unificación Datasets y Búsqueda de Lyrics

Causas



Variantes que la normalización no cubre

Títulos con orden distinto de colaboradores, feat., typos, remasters, entre otros.



Canciones poco conocidas

De las 4495 canciones presentes en el dataset de Spotify, 3146 son poco conocidas por lo que hay muchos temas poco documentados en APIs públicas.

Solución Implementada

Spotify no conoce el id de Genius y viceversa. El único vínculo es el par (artista, título), que difiere entre fuentes: acentos, mayúsculas, sufijos como "- Remastered 2011".
Una igualdad exacta de strings fallaría en casi todos los casos.

Fuzzy Join

Canonicalizamos (acentos, puntuación, sufijos, artista principal) y sondeamos Genius en **tres niveles de estrictez decreciente**: exacto, artista exacto con título difuso, y ambos difusos.

Backfill de Lyrics

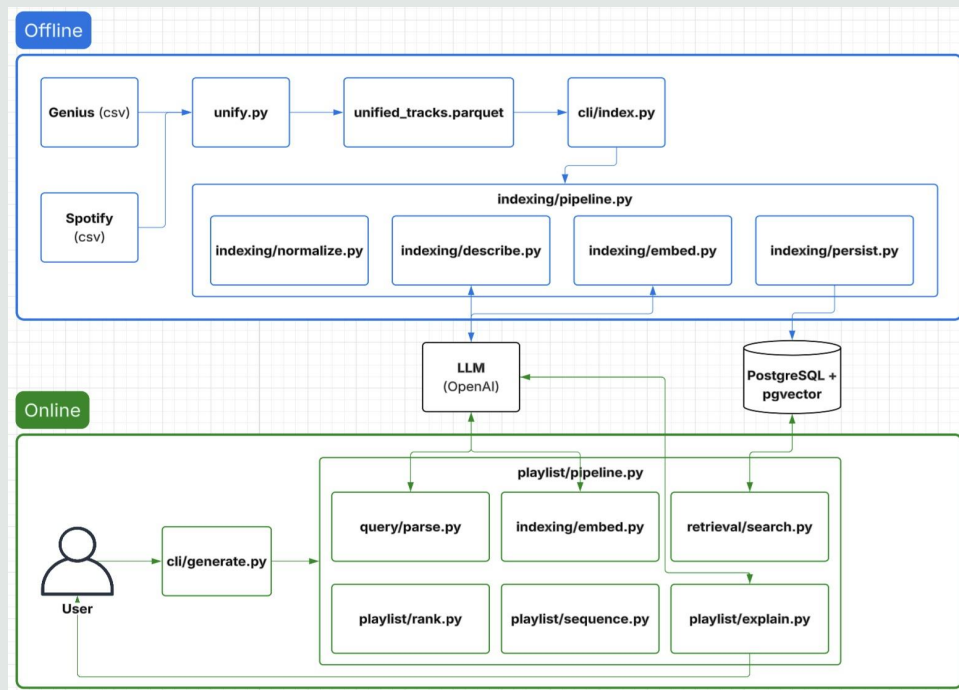
Sobre las filas sin letra contra una API externa. Reanudable mediante caché en disco por **track_id**. Solo los IDs que agregan letra entran a re-indexación, **evitando recalcular embeddings del catálogo completo**.

04

Decisiones de Implementación

Decisiones clave junto con sus justificaciones

Arquitectura del RAG



Conservar Canciones sin Letra

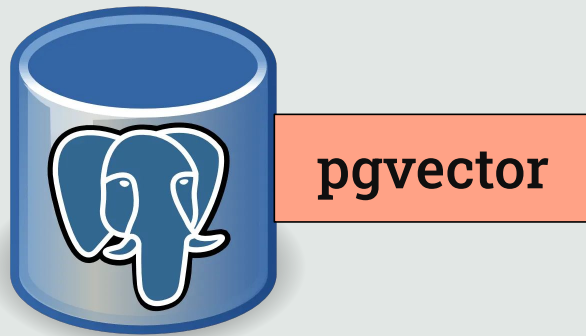
Descartar ~55% del dataset reduciría mucho el catálogo

```
SELECT COUNT(*)  
FROM tracks  
WHERE lyrics IS NULL;
```

	count bigint 
1	2424

Especialmente en el catálogo de canciones poco conocidas, donde muchas veces no se tienen letras. Por este motivo, el pipeline utiliza dos prompts distintos para generar la descripción semántica: uno para canciones con lyrics y otro adaptado para tracks sin letras.

Base de Datos Elegida



Se eligió PostgreSQL + pgvector por su simplicidad, integración entre SQL y búsqueda vectorial, y menor complejidad operativa.

Embedding de la Descripción

Offline:

Letra

"You can dance, you
can jive
Having the time of
your life..."

Audio

Features

"danceability: 0.543
energy: 0.87
..."

Metadata

"track_artist: ABBA
track_name:
Dancing Queen
..."

Un embedding de letra
completa o de una parte
se acerca a contenido
lírico.

LLM

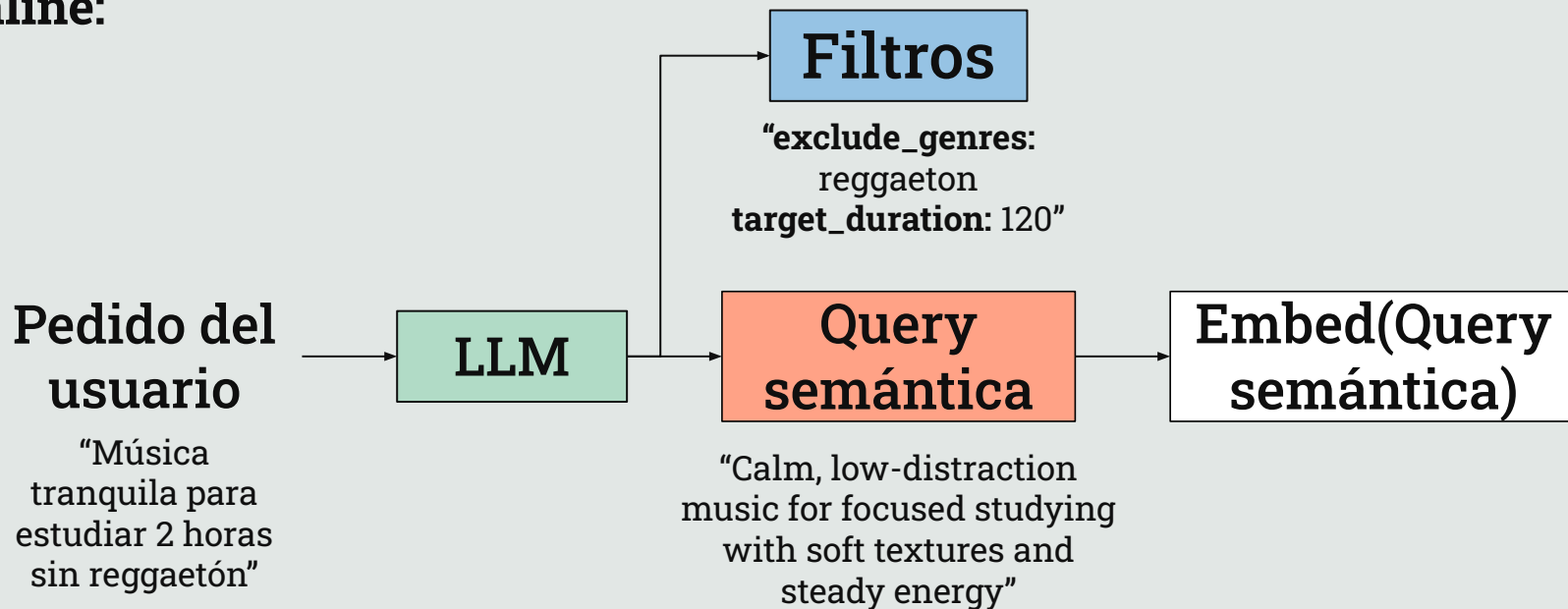
Description

Embed(Description)

"A quintessential disco anthem,
'Dancing Queen' radiates joy with its
upbeat tempo and infectious energy..."

Embedding del Pedido del Usuario

Online:



Métricas Elegidas

Context Precision

¿Los tracks recuperados son relevantes y están bien rankeados?
Toma tracks del pool de retrieval y un LLM marca por cada uno si es relevante o no para la query.

Context Recall

¿El pool recuperado cubre los requisitos del caso de prueba?
Cada caso tiene **reference_statements** (4 afirmaciones). Un LLM juzga si el bloque de tracks permite satisfacer cada afirmación.

Faithfulness

¿La explicación inventa hechos que no están en las canciones elegidas?
Un LLM extrae claims atómicos de la explicación. Luego, marca si cada claim es atribuible a metadata/description de los tracks de la playlist.

Answer Relevance

¿La respuesta (playlist + explicación) responde al pedido original?
Un LLM genera N pedidos estilo usuario a partir de la respuesta y se calcula el promedio de cosenos.

Más Métricas Elegidas

Duration Adherence

¿La duración final se aproxima a **target_duration_minutes**?

Exclusion Adherence

¿Se respetaron **exclude_artists** / **exclude_genres**?

Artist Diversity

Ratio artistas únicos / total tracks

Genre Diversity

Entropía normalizada de subgéneros

Estas nuevas métricas complementan a las anteriores. El sistema puede tener **alta faithfulness** pero una **duración incorrecta**, o buen **retrieval** con **exclusiones mal aplicadas**.

05

Resultados Obtenidos

Ejemplos de uso

Resultados Obtenidos

7 casos sobre el catálogo indexado. Métricas RAGAS más adherencia y diversidad.

Categoría	Métrica (RAGAS / Obj)	Valor Medio	Estado
Retrieval	Context Precision / Recall	0.81 / 0.82	● Sólido
Generación	Faithfulness	0.59	● A mejorar
Generación	Answer Relevance	0.75	● Estable
Restricciones	Exclusiones / Duración	1.00 / 0.99	● Perfecto
Diversidad	Artist / Genre Diversity	0.95 / 0.93	● Alta

Resultados Globales

Análisis del Peor Caso:

“indie and alternative tracks I might not know, eclectic mix for an hour”

Métrica de Evaluación	Valor Obtenido	Media Global	Estado
Context Precision	0.671	0.812	▼ Inferior
Context Recall	1.000	0.821	▲ Excelente
Faithfulness	0.333	0.594	▼ ▼ Crítico
Answer Relevance	0.724	0.753	≈ Estable
Duration Adherence	0.908	0.987	▼ Corto

Análisis del Peor Caso:

“indie and alternative tracks I might not know, eclectic mix for an hour”

Explicación generada:

"This playlist offers an eclectic mix of indie and alternative tracks that you might not have encountered before, creating a laid-back yet engaging vibe perfect for an hour of discovery. Each selection aligns with your request for lesser-known artists... The flow is smooth, transitioning between upbeat and mellow tracks while maintaining a cohesive sound throughout."

#	Claim	¿Atribuible?	Por qué
1	"eclectic mix of indie and alternative"	✗	13/13 son australian indie — un solo nicho, no ecléctico
2	"lesser-known artists"	✓	13/13 <code>tier=obscure</code> — soportado por la data
3	"laid-back yet engaging vibe"	✗	6/13 <code>energy=high</code> , bpm hasta 169 → no es "laid-back"
4	"perfect for an hour of discovery"	✗	dura 46 min , no 60 (<code>dur=0,908</code>) — el claim contradice el dato
5	"smooth flow, upbeat↔mellow transitions"	✓	el sequencer sí ordena por transición (parcialmente real)
6	"fresh and dynamic listening experience"	✗	subjetivo, sin anclaje en metadata

Las 6
claims y
su
veredicto

Análisis del Peor Caso:

“indie and alternative tracks I might not know, eclectic mix for an hour”

0.333

Faithfulness Score

Causa de la caída

El Juez RAGAS invalidó 4 de los 6 claims generados por el LLM. El problema no es la alucinación de tracks (que existen y son correctos), sino la **adjetivación no anclada**.

El LLM empleó prosa de marketing ("ecléctico", "laid-back") que contradice la metadata técnica de los tracks recuperados.

“Rock classics for 1-hour run”

Why this playlist

This playlist features a high-energy selection of classic rock tracks designed to keep you motivated throughout your 1-hour run. With iconic anthems from artists like AC/DC, Van Halen, and The Clash, the vibe is both invigorating and nostalgic, perfect for pushing your limits. The flow is seamless, transitioning from upbeat hits to powerful ballads, ensuring a diverse yet cohesive listening experience.

15 tracks · ~60 min · 80 candidates retrieved

#	Title	Artist	Mood	Energy	Tempo	Duration	Why	Spotify
1	Have You Ever Seen The Rain	Creedence Clearwater	nostalgic	high	116	2:40	semantic match (0.47); energy=high; classic rock	open ↗
2	Murder on the Dance Floor - triple j Like A Version	Royel Otis	joyful	high	123	3:01	semantic match (0.41); energy=high; indie rock	open ↗
3	Run To You	Bryan Adams	energetic	high	127	3:53	semantic match (0.44); energy=high; classic rock	open ↗
4	Hungry Like the Wolf - 2009 Remaster	Duran Duran	energetic	high	128	3:41	semantic match (0.41); energy=high; synth-pop	open ↗
5	Carry on Wayward Son	Kansas	uplifting	high	127	5:23	semantic match (0.42); energy=high; classic rock	open ↗
6	Song 2 - 2012 Remaster	Blur	energetic	high	130	2:01	semantic match (0.41); energy=high; alternative rock	open ↗
7	Jump - 2015 Remaster	Van Halen	energetic	high	130	4:02	semantic match (0.42); energy=high; classic rock	open ↗
8	The Final Countdown	Europe	energetic	high	118	5:10	semantic match (0.40); energy=high; arena rock	open ↗
9	Black Betty	Ram Jam	energetic	high	118	3:58	semantic match (0.37); energy=high; southern rock	open ↗
10	Hells Bells	AC/DC	angry	high	107	5:12	semantic match (0.39); energy=high; hard rock	open ↗

Breve explicación de la playlist creada.

Descripción semántica y lista generada con género y tiempo adecuado.

“Rock classics for 1-hour run”

El usuario puede ver la query que utiliza el modelo para obtener los resultados.

What the model understood

```
{
  "semantic_query" : "A collection of classic rock songs to energize my 1-hour run."
  "target_duration_minutes" : 60
  "energy_levels" : [
    0 : "high"
  ]
  "include_genres" : [
    0 : "rock"
  ]
}
```

“Rock classics for 1-hour run but no Creedence or AC/DC”

Las restricciones en la query ingresada se transforman en filtros adecuados.

▼ What the model understood

```
▼ {  
  "semantic_query" : "Rock classics to energize my 1-hour run, avoiding Creedence and AC/DC."  
  "target_duration_minutes" : 60  
  ▼ "energy_levels" : [  
    | 0 : "high"  
    |  
  ]  
  ▼ "include_genres" : [  
    | 0 : "rock"  
    |  
  ]  
  ▼ "exclude_artists" : [  
    | 0 : "Creedence"  
    | 1 : "AC/DC"  
    |  
  ]  
}
```

06

Conclusiones

Últimas palabras

Conclusiones

El proyecto nos permitió destacar un caso de uso claro de RAG y cómo se diferencia de simplemente consultar un LLM.

Las decisiones de mayor impacto fueron aquellas ligadas a la ingeniería de datos y robustez: la union, curado, resumibilidad/idempotencia del indexado.

**¡Muchas
Gracias!**

